

Introduction à la programmation MPI

TD 4 -

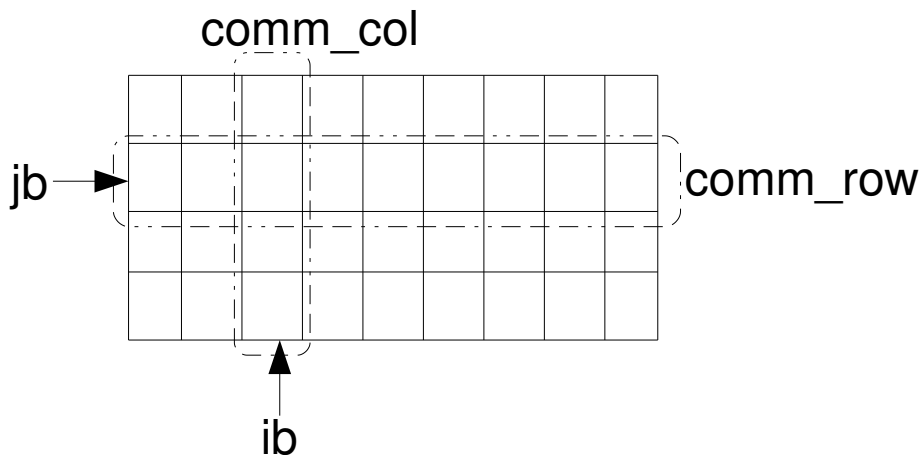
Types dérivés, Communicateurs et Réductions non-bloquantes

Exercice I : Découpage par bloc d'une matrice pleine

On veut distribuer une matrice pleine en la découpant par bloc.

Dans cet exercice, nous avons autant de blocs que de processus MPI (un bloc = un processus MPI).

Un bloc est repéré par ses coordonnées (ib, jb) et le nombre total de blocs est égal à $N_{ib} \times N_{jb}$. La relation entre le rang MPI et les coordonnées (ib, jb) est la suivante : $\text{rang} = ib \times N_{jb} + jb$.



1. A l'aide de la fonction `MPI_Comm_split`, créer 2 communicateurs par processus :
 - `comm_row` : ensemble des processus de la même rangée ;
 - `comm_col` : ensemble des processus de la même colonne ;
2. Comment récupérer N_{ib} et N_{jb} à l'aide d'une fonction MPI ?
3. Comment récupérer (ib, jb) à l'aide d'une fonction MPI ?

Exercice II : Réduction sur nombres complexes

On considère un tableau distribué de nombres complexes (de type `complex_t`).

Partie 1 :

On veut déterminer l'élément qui a la plus grande norme (la fonction `norm_compl` existe déjà).

1. A partir de la fonction `MPI_Type_contiguous` construire un type `mpi_cpl` correspondant à une variable de type `complex_t`
2. Créer l'opérateur MPI `op_norm_max` qui permettrait de calculer le nombre complexe avec la plus grande norme lors d'une réduction
3. Compléter la fonction :

```
void calc_norm_max(  
    complex_t *tab, int n,  
    MPI_Op op_norm_max,  
    MPI_Datatype mpi_cpl,  
    complex_t *cpl_max)
```

qui retourne dans `*cpl_max` la valeur du nombre complexe de plus grande norme parmi le tableau distribué (`tab`, `n`).

Partie 2 :

A présent on veut à la fois obtenir le nombre complexe de plus grande norme mais également le rang du processus qui détient initialement ce nombre. Pour ce faire, le type `complex_loc_t` est défini.

1. A partir des fonctions `MPI_Type_create_struct` et `MPI_Get_address` construire un type `mpi_cpl_loc` correspondant à une variable de type `complex_loc_t`
2. Créer l'opérateur MPI `op_norm_max_loc` qui permettrait de calculer, lors d'une réduction, le nombre complexe avec la plus grande norme ainsi que son rang
3. Compléter la fonction :

```
void calc_norm_max_loc(  
    complex_t *tab, int n,  
    MPI_Op op_norm_max_loc,  
    MPI_Datatype mpi_cpl_max_loc,  
    complex_loc_t *cpl_max_loc)
```

qui retourne dans `*cpl_max_loc` la valeur et le rang du nombre complexe de plus grande norme parmi le tableau distribué (`tab`, `n`).

Exercice III : Charges déséquilibrées

Dans une boucle, chaque processus MPI appelle une fonction `work()` dont la charge de travail est aléatoire (entre 1 et 10 s). Le programme doit s'arrêter quand la charge totale de travail dépasse une valeur donnée en argument en ligne de commande.

Dans le fichier `unbal_workload/exercice/exo_unbal_workload.c`, un appel à `MPI_Allreduce` est réalisé à chaque itération pour connaître la charge de travail totale déjà réalisée et la comparer à la charge totale à atteindre.

- 1) Compiler `exo_unbal_workload.c`, exécuter sur 8 processus MPI avec 400 pour charge totale (à passer en argument). Mesurer les temps min et max.
- 2) Proposer une nouvelle implémentation à base de communication non bloquante pour minimiser le temps total de restitution.